

- PART I · 基础
- PART II · Skills
- PART III · Subagents
- PART IV · Hooks
- PART V · 组合
- PART VI + 附录

本 PART 章节

- 22. Ch 22 · Distribution, Team Adoption, and the Marketplace
- A. Appendix A · Troubleshooting and Debugging
- B. Appendix B · Hook Events Quick Reference

原书第 217 页 · PART SIX / CHAPTER TWENTY-TWO 起始

217

PART SIX · 第六部分

Shipping, Sharing, and Evolving

发布、分享、演进

The environment built once is the foundation. The environment maintained over time is the asset.

环境"被搭起来一次"是地基;"随着时间被维护"才是资产。

CHAPTER TWENTY-TWO · 第二十二章

Distribution, Team Adoption, and the Claude Code Marketplace

分发、团队采纳,以及 Claude Code 市场

"The garden you tend for years is not the garden you planted. It is the garden you allowed to become itself, with help."

— a gardener, anonymous

"你照看了多年的花园,已经不是你当初种下的那个花园了。它是你在助力之下,被允许成为自己的那个花园。"

—— 一位园丁(匿名)

This is the closing chapter of the book. The work of building has been the focus of every part up to now. The work of evolving is the focus of this one. A Claude Code environment that has been built and never touched again will degrade slowly, as the developer's work shifts and the environment fails to shift with it. A Claude Code environment that has been treated as a living artifact, gardened regularly, shared with others, and integrated into the broader ecosystem of available tooling, will continue producing benefits for years, and those benefits will compound in ways that no static environment can match. The chapter ahead is about that gardening.

这是本书的收尾一章。到此为止的每一部分,重心都落在"搭"上;本章的重心是"演化"。一份 Claude Code 环境,搭好之后再也不动——会随着时间慢慢失能:开发者的工作在迁移,环境却没能跟着迁移。一份 Claude Code 环境,如果被当作活的工件对待、定期打理、与他人共享、并接入更广的工具生态——则会在多年里持续产出效益,这种效益会以任何静态环境都匹配不了的方式复利。本章讲的就是"打理"。

Begin with the first concern, which is version control. The `.claude` directory is part of a project's contract for how work happens in the project. It belongs in source control, alongside the source code. Every project the developer works in should have its `.claude` directory committed to its repository, with changes reviewed in pull requests like any other code change. This is not an aesthetic recommendation. It is the mechanism by which configuration travels with the code, by which teammates inherit the environment automatically when they clone, and by which the configuration evolves in a reviewed, accountable, reversible way. Projects that treat their `.claude` directories as ephemeral local configuration fail to capture most of the value the architecture is capable of providing. Projects that treat them as first-class engineering artifacts capture the full value, and the difference compounds over months and years.

第一件事,版本控制。`.claude` 目录是项目"工作如何发生"那份契约的一部分——它应当和源代码一起住进版本控制。开发者工作的每一个项目,都应当把 `.claude` 目录提交进仓库,并按"和其

他代码变更一样"的方式在 PR 中 review。这不是审美建议。它是"配置随代码走"的机制——队友 clone 时自动继承环境的机制——配置以一种"可 review、可追责、可回滚"的方式演化的机制。那些把 `.claude` 目录当"一次性本地配置"对待的项目,捕获不到这套架构本可提供的大部分价值。那些把它当一等工程工件对待的项目,捕获到的是它的全部价值;而这种差别会在一年又一年里复利下去。

The user-level configuration, which lives in the home directory and applies across every project the developer opens, deserves the same treatment, in a slightly different form. The user-level `.claude` directory should also be version-controlled, but typically in a personal repository rather than in any specific project's repository. The personal repository becomes a record of how the developer's tooling has evolved over time. It can be cloned to a new machine when the developer moves laptops. It can be shared with peers as a starting point when they ask how she has set up her environment. It can be referenced years later as a working portrait of how the developer's craft has grown. The version control of the user-level configuration is a small habit, but the developers who adopt it consistently end up with substantially more durable tooling than those who treat their home directories as ephemeral.

User 级别的配置——住在 home 目录、对开发者打开的每一个项目都生效——也值得同等对待,只是形态略有不同。User 级别的 `.claude` 目录也应当被版本控制,但通常是放一个"个人仓库"里,而不是塞进某个具体项目的仓库。这个个人仓库,就成了"开发者工具如何随时间演化"的一份记录。开发者换笔记本时,可以直接把这份配置 clone 到新机。当同事问"你怎么配的环境"时,她可以把这份配置当起点分享出去。几年后,这份仓库还可以被翻出来,作为"她的手艺是如何成长"的一份可工作画像。给 user 级配置做版本控制,只是一条小习惯;但稳定采纳这条习惯的开发者,手里的工具会比那些把 home 目录当一次性用品的人耐久得多。

← 前一页

后一页 →

A natural extension of version control is the practice of writing meaningful commit messages for `.claude` directory changes. A commit that says "added prettier hook" tells the developer in six months what she changed and why. A commit that says "config update" tells her nothing. The same discipline that produces good commit messages for source code applies to commit messages for configuration, and

the same skill the developer has installed for source-code commits, the `commit-message` skill from Chapter Five, applies cleanly to configuration commits as well. The configuration is code, in every meaningful sense, and treating it as code produces the consistency that long-term maintenance requires.

版本控制顺理成章的下一步,就是"为 `.claude` 目录的变更也写有意义的 commit 消息"。一条写着"added prettier hook"的 commit,让六个月后的你一眼看明白改了什么、为什么改;一条只写着"config update"的,什么都看不出来。"为源代码写好 commit 消息"那条纪律,直接套到"为配置写 commit 消息"上即可;而开发者为源代码 commit 装的那条 skill——第五章的 `commit-message` skill——也能干干净净地用在"配置 commit"上。配置在任何有意义的层面上都是代码;把它当代码对待,正是产生"长期维护所需的一致性"的那条路。

Move to the second concern, which is sharing. A skill, subagent, or hook that has proven valuable in one developer's environment is often valuable in many others. The mechanism for sharing is straightforward. The component is a file or a small folder. The file or folder can be copied into another developer's `.claude` directory, or distributed through a public repository that other developers clone, or installed through one of the package managers that have begun to emerge around the Claude Code ecosystem. The friction of sharing is low, and the benefit of receiving someone else's well-designed components is substantial.

第二件事,分享。一条 skill、一个 subagent、一个 hook——在某个开发者环境里被证明是有价值的,常常在很多其他人的环境里也值钱。分享的机制很直接:组件本身就是一个文件或一个小目录;这个文件/目录可以被拷到另一个开发者的 `.claude` 目录里,可以通过一个公共仓库被其他人 clone 走,也可以通过 Claude Code 生态里正在冒出来的某个包管理器来装。分享的摩擦很低;拿到别人设计良好的组件,所获得的收益却很可观。

The Marketplace · 市场

The Claude Skills Marketplace is the most prominent public channel for sharing. Anthropic has begun curating skills produced both by its own engineering team and by third-party developers and organizations, with skills available for installation through a small command-line interface. The marketplace is in its early days as of this writing, and the available catalog is modest, but the trajectory is clear. Skills that solve common problems will continue to accumulate in the marketplace, and the developer who watches the marketplace will find ready-made solutions for problems she might otherwise have built skills for from scratch. The right relationship with the marketplace is to use it as a starting point rather than as a complete library; off-the-shelf skills rarely fit a specific developer

preferences perfectly, but they often save substantial design time when used as a base that the developer adapts.

Claude Skills Marketplace 是分享这件事上最显眼的公共渠道。Anthropic 已经开始"策展"由自家工程团队、以及由第三方开发者和组织产出的 skills——通过一个小型的 CLI 就能装。在本书写作的当下,这个市场还处在早期,可用目录也不算丰富,但方向已经很清晰。能解决常见问题的 skills 会持续在市场里沉淀;而关注市场的开发者,会为"那些她本来要从头搭起 skill 的问题"找到现成的解。和市场打交道的正确姿态,是把它当作"起点",而不是"完整的库"——现成的 skill 很少能完美匹配某一具体开发者的偏好;但当作"被自己再去适配"的基础时,常常能省下可观的"从零设计"时间。

Subagents and hooks have not yet developed marketplace structures of equivalent maturity, but the underlying mechanism is the same. Public

Subagents 和 hooks 还没有发展到同等成熟度的市场结构,但底下的机制是同一种。公共——

← 前一页

后一页 →

repositories of subagent definitions and hook scripts are appearing in greater numbers, often shared on GitHub under permissive licenses, and the developer who participates in this informal exchange both contributes to and benefits from the collective body of work. Sharing publicly is also a way of making one's own work better, because the act of writing a component that strangers will read forces a level of clarity that internal-only components do not require.

subagent 定义、hook 脚本的公共仓库正在越来越多地出现——通常以宽松许可证托管在 GitHub 上。参与这种非正式交换的开发者,既为这份集体工作添砖,也从这份集体工作里获益。公开分享,还是一种"让自己手上的活变得更好"的方式:为陌生人写一个组件这件事本身,就逼出一种"只写给内部人"时所不需要的清晰度。

A third concern is monetization. Some developers will, naturally, want to sell their work. The economics of this are still settling in 2026, but several patterns have emerged that look likely to persist. Skill packs targeted at specific niches, sold as one-time purchases or as subscriptions, are appearing on platforms like Gumroad and Lemon Squeezy. Subagent libraries customized for specific industries are being licensed by consultancies to their clients. Configuration packages that combine skills, subagents, and hooks into complete environments for specific roles are being sold as productivity products. The market is small but growing, and the developers who build genuinely high-quality components have a real path to monetization that did not exist a few years ago.

第三件事,商业化。一些开发者自然会想把自己手上的活卖出去。这件事的经济形态在 2026 年仍处于成形期,但已经浮现出几种看上去会持续存在的范式:针对特定 niche 的 skill pack,以"一次性买断"或"订阅"的形式,在 Gumroad、Lemon Squeezy 这类平台上出现;按特定行业定制的 subagent 库,正被咨询公司授权给它们的客户;把 skills、subagents、hooks 组合成"针对特定角色的完整环境"的配置包,正作为生产力产品在出售。这个市场还小,但在增长;真正能造出高质量组件的开发者,面前摆着一条几年前还不存在的商业化通路。

The right relationship with monetization, for most developers, is to think of it as an option rather than as an objective. Building components for one's own work is the primary purpose. If, over time, certain components turn out to have wider value, the path to selling them is open. Building components for sale from the start, before any internal use has validated their quality, is harder, and the components produced this way often turn out to be less useful than components that emerged from real work. The advice that emerges is to build first, sell later, and to let the components prove themselves in actual use before any commercial considerations enter the picture.

对大多数开发者来说,和"商业化"打交道的正确姿态,是把它当一个"选项",而不是"目标"。为自己的工作搭组件,才是第一目的;如果在过程中,某些组件被发现具有更广的价值,把它们卖出去的路是敞开的。一上来就奔着"卖"去搭组件——还没经过自己真实使用验证的那种——往往更难;这样搭出来的组件,实用度常常比不上"从真实工作中长出来"的那些。从中提炼出的建议是:先搭,后卖;让组件先在真实使用里证明自己,再让商业考虑入局。

Move to the fourth concern, which is team standardization at scale. The case study in Chapter Twenty showed a small team adopting a shared configuration. The same pattern scales to larger organizations, but with additional considerations. A team of fifty engineers has more diversity of working style than a team of five, and a configuration that works for the entire team needs to be either narrow enough to fit

everyone or layered carefully so that the shared layer represents the team's genuine common ground while individual variation lives in personal user-level configuration.

第四件事,规模化下的团队标准化。第二十章那个案例展示的是"小团队采纳一份共享配置"。同一套范式可以放大到更大的组织,但要补一些新的考量。一个 50 人的工程师团队,在"工作风格多样性"上远超 5 人团队;要让一份配置对全队都管用,要么它得"足够窄",能装下所有人,要么它得"仔细分层":共享层只承载团队真正的公约数,个人差异则放在每个人的 user 级配置里。

← 前一页

后一页 →

Team Standardization at Scale · 规模化下的团队标准化

The pattern that has worked best in practice is to keep the shared project-level configuration tight and conservative, including only the components that genuinely apply to all engineers in the project, and to encourage individual engineers to maintain their own personal user-level configuration on top of it. The shared layer captures conventions, security guarantees, and project-specific knowledge. The personal layer captures preferences, role-shaping, and personal-style choices. The combination produces a personalized environment for each engineer that nonetheless honors the team's common standards. The split also makes the shared layer easier to evolve, because changes to the project-level configuration affect only what is genuinely shared, not the personal preferences that should not be overridden.

实践中真正管用的范式是:把"项目级共享配置"做得紧凑而克制——只装那些"对项目里所有工程师都适用"的组件;同时鼓励每位工程师在自己 user 级别的个人配置里再长一层。共享层抓的是规约、安全保证、项目特有知识;个人层抓的是偏好、角色塑形、个人风格选择。两者合起来,给每位工程师一个"既尊重团队公约数、又带个人色彩"的环境。这种切分也让共享层更容易演进——因为对 project 级配置的改动,只影响"真正属于共享"的那部分,而不会动到那些"不该被覆盖"的个人偏好。

A fifth concern is the long-term evolution of the developer's environment. After a few years of working with the architecture, the developer's `.claude` directory becomes a substantial artifact. It might contain forty skills, fifteen subagents, and a dozen hooks across her various projects and her user-level configuration. The maintenance posture for an environment of this size is different from the posture for a small new environment. The questions shift from what to add to what to keep, what to remove, and what to refactor. A quarterly review becomes a useful discipline. The developer sets aside an hour every three months to walk through her configuration, identify components that have not been used recently, refine components that have been used heavily, and remove components that have outlived their purpose. The hour pays for itself many times over in the months that follow, because it keeps the environment from accumulating dead weight that would otherwise quietly degrade the experience.

第五件事,是开发者环境的长期演化。在和这套架构共处几年之后,开发者的 `.claude` 目录会长成一份可观的工件——可能跨多个项目和 user 级配置,装着 40 个 skills、15 个 subagents、十几个 hooks。维护这种体量的环境,姿态和"新搭起来一份小型环境"是不同的:问题从"加什么"转向"留什么、删什么、重构什么"。一种有用的纪律是"季度回顾"——每三个月花一个小时,把自己的配置过一遍:标出最近没用过的组件、精修被频繁用到的组件、删掉已经走过其生命周期的组件。这一个小时会在随后的几个月里以数倍的回报被挣回来——因为它让环境不至于积累"死重",而那些死重如果不清,会悄悄拖垮整个体验。

A sixth concern is the relationship between the developer's Claude Code environment and the broader ecosystem of AI-assisted development tools that is forming. Skills, in particular, are increasingly being adopted by other tools. The `SKILL.md` format is being honored by Cursor, by various open-source IDE extensions, and by several command-line tools that are not Claude Code. A skill written for Claude Code today often works without modification in three or four other tools. This cross-compatibility is not accidental; it reflects the convergence of the industry around a small set of

第六件事,是开发者的 Claude Code 环境与正在形成的、更广的"AI 辅助开发工具"生态之间的关系。Skills 尤其正在被越来越多的其他工具采纳。`SKILL.md` 这套格式,正在被 Cursor、被各种开源 IDE 扩展、被若干"不是 Claude Code"的 CLI 工具所尊重。今天为 Claude Code 写的一条 skill,常常不加修改就能在三四个其他工具里跑。这种跨工具的兼容性不是偶然的——它反映了整个行业正在向一小套

standards, with `SKILL.md` emerging as one of those standards. The implication for the developer is that her investment in skill design pays dividends not just inside Claude Code but across whatever set of AI-assisted tools she ends up using over the years. The skills travel. The environment, in this sense, is more durable than the specific tool the environment was built for.

标准收敛——`SKILL.md` 正在成为其中之一。对开发者的含义是:她在 skill 设计上的投入,收益不只是落在 Claude Code 内部,而是会跨她未来多年里会用到的各种 AI 辅助工具,持续产生回报。Skills 是会"旅行"的。从这个意义上,环境比它当初为之搭建的那个具体工具更耐久。

A seventh concern is what the developer can expect from the ecosystem in the months and years ahead. The trajectory of Claude Code, and of similar tools, is toward more sophisticated automation, more capable subagents, and more richly composable hook systems. New lifecycle events will appear. New tool restrictions will become possible. New patterns of subagent communication will emerge. The developer who has internalized the architecture in this book will be well-positioned to absorb each of these changes as they arrive, because each will fit into the framework rather than disrupting it. The framework itself is not tied to any specific feature set. It is tied to the durable insight that capability, specialization, and enforcement are three different concerns that benefit from being handled by three different mechanisms. That insight will remain valid as long as AI-assisted development continues to be useful, which is to say, indefinitely.

第七件事,是开发者可以预期未来几个月到几年里,生态会朝哪些方向走。Claude Code 以及同类工具的轨迹,指向更复杂的自动化、更能干的 subagent、组合性更丰富的 hook 系统。会有新的生命周期事件出现;会有新的工具限制成为可能;会有新的 subagent 通信范式浮现。已经把本书这套架构内化于心的开发者,会处在很好的位置去吸收这些变化——因为每一项变化都会"嵌进"这套框架,而不是去颠覆它。框架本身并不绑死在任何具体的功能集上;它绑死在一条耐久的洞察上——能力、专门化、强制,是三件不同的事,由三种不同的机制去处理,这件事整体上有利。这条洞察,会在 AI 辅助开发仍然有用的整个时期里持续成立;换言之,会无限期地成立。

A reflection on what the book has tried to do. The book has tried to teach a way of relating to a tool that, for most of its users, is treated as a chat interface. The way of relating that the book has taught is more like the way an engineer relates to a well-configured editor or a well-tuned shell: as a substrate that can be bent to fit the engineer's work, that rewards patient configuration with compounding returns, and that becomes, over time, an extension of the engineer's craft rather than merely a tool she uses. The shift in posture is what matters most. The specific skills, subagents, and hooks that the book has shipped are starting points. The shift in posture, once made, persists across every future tool the developer encounters, because it is a way of thinking about tooling rather than a fact about any specific tool.

回头看看,这本书究竟在尝试做什么。它在尝试教一种"和工具相处的方式"——这个工具,在它的大多数用户手里,被当成了"一个聊天界面"。本书教的那种相处方式,更接近于"工程师和一款配得好的编辑器、和一副调得顺手的 shell"之间的关系:把它当作"可以弯到自己工作形状"的底层;用耐心的配置去换复利的回报;让它随着时间,变成工程师手艺的延伸,而不是她用一下就放下的工具。这种姿态上的转变,才是关键。本书走过的那些具体 skills、subagents、hooks,只是起点。姿态一旦转换,就会在开发者未来遇到的每一个工具里延续——因为它是一种"关于工具该怎么想"的思维方式,而不是关于任何一个具体工具的事实。

The Community of Practice · 实践共同体

A second reflection concerns the community of practice that is forming around the architecture this book has been teaching. The community is real, growing, and largely informal. Developers are sharing their configurations

第二条反思,是关于"围绕本书所教架构正在形成中的实践共同体"。这个共同体是真实存在的、正在成长的、很大程度上是非正式的。开发者们在分享自己的配置——

← 前一页

后一页 →

on GitHub, writing blog posts about their patterns, recording short videos of their environments in action, and trading components in private Slacks and Discords. The community is not centralized; there is no single forum that captures all of it, and no single voice that speaks for it. The decentralization is a strength, because it means that good ideas surface from many different directions and the practice evolves through real use rather than through pronouncement. The reader who wants to participate has many entry points. She can publish her own configuration. She can contribute components to the marketplace. She can write about what has worked for her. She can simply adopt other developers' ideas and incorporate them into her own environment. The community rewards participation, even modest participation, with both reciprocal benefit and a sense of belonging to something that is moving in real time.

在 GitHub 上分享、写 blog 总结自己的范式、录短视频展示自己环境实际跑起来的样子、在私有的 Slack 和 Discord 里交换组件。这个共同体不是中心化的——没有哪个单一论坛能装下它全部,也没有哪个单一声音能代表它。分散正是它的力量所在:好点子会从很多不同方向冒出来,这套实践是经由"真实使用"演化的,而不是靠"权威宣告"。读者想参与,入口很多:可以发布自己的配置;可以向市场贡献组件;可以写文章讲讲自己踩过什么;也可以只是把别人的想法接过来,嵌进自己的环境。这个共同体回报"参与"——哪怕是 modest 的参与——以双向的收益、以及对"自己在参与一件正在实时向前推进的事"的归属感。

A third reflection concerns contribution upstream, to Anthropic and to the broader tooling ecosystem. The patterns this book has been teaching are not fixed. They are evolving, and the people who use them daily are the most likely to discover the patterns that should evolve next. A developer who notices a recurring failure mode in subagent design, who finds a hook event that does not exist but should, who realizes that some category of skill would benefit from a feature the format does not yet support, has the standing to surface that observation. The right channels include filing issues on the relevant tools' GitHub repositories, writing publicly about the observation, and engaging with the developer-relations teams who watch the community. Many of the features that have shipped in Claude Code over the past year originated in observations from real users. The reader who has worked through this book and developed a serious practice has the standing to contribute to the next round of those observations, and the contribution is not abstract: it shapes the tools she will be using a year from now.

第三条反思,是关于"向上游——Anthropic 和更广工具生态——做贡献"。本书一直在教的范式不是定死的;它们在演化;而每天都在用这些范式的人,最有可能发现"下一步该往哪里演化"。一位开发者,如果在 subagent 设计中注意到反复出现的失效模式,或者发现"某个 hook 事件本该有但还没有",或者意识到"某一类 skill 会受益于一个这个格式目前还不支持的能力",她就有资格把这种观察

推到台面上。合适的渠道包括:在相关工具的 GitHub 仓库提 issue,把这些观察写出来公开发表,以及和"注视着这个共同体"的开发者关系团队互动。过去一年里 Claude Code 推出的许多功能,源头就是真实用户提出来的观察。认真读完本书、且已形成严肃实践的读者,有资格去贡献下一批观察;而这种贡献不是抽象的——它会塑造她明年会用到的那种工具。

A fourth reflection concerns the longevity of the architecture itself. The framework in this book has been described in terms of one specific tool, Claude Code, and one specific moment in time, the spring of 2026. The tool will change. The moment will pass. The framework, however, is durable, because it is rooted in a deeper truth about how AI-assisted development naturally divides into knowledge, specialization, and enforcement. Any future tool that handles AI-assisted work will, by the structure of the work itself, present similar concerns, and the framework will translate into

第四条反思,是关于"这套架构本身的耐久性"。本书里讲的框架,是围绕一个具体工具(Claude Code)、一个具体时点(2026 年春)展开的。工具会变;时点会过去。但框架本身是耐久的,因为它扎根于一条更深的真理:AI 辅助开发天然地分裂为"知识、专门化、强制"这三块。任何一个未来做 AI 辅助工作的工具,只要工作本身的结构没变,就会呈现同样的关切;而这套框架,会被翻译成——

← 前一页

后一页 →

whatever vocabulary that future tool adopts. A reader who has internalized the architecture is, in this sense, learning a posture that will outlast the specific tool the book was written for. The skills she develops in shaping her Claude Code environment are the same skills she will use to shape whatever tool replaces it, and whatever tool replaces that, indefinitely. The investment is not in Claude Code specifically. The investment is in the practice of engineering one's relationship with one's primary tooling, and the practice transfers.

那个未来工具所采用的任何词汇表。在这个意义上,一位把架构内化于心的读者,所学的是一种姿态——这种姿态会比"本书为之写作的那款具体工具"更耐久。她在塑造自己 Claude Code 环境时练就的这些技能,也是她用来塑造"取代它的那款工具",以及"再取代那款的工具"——一直接力下去

——的同一些技能。这份投入,不是投给"Claude Code"这一个具体对象;而是投给"把自己和主力工具的关系工程化"这门实践。这门实践是可迁移的。

The reader who has worked through the book has, at this point, the framework, the patterns, and the practical libraries to build a substantial Claude Code environment for her own work. The next steps are her own. She will install the components from the chapters that fit her work most directly. She will adapt the others to her own preferences. She will build her own components for the frictions the book did not anticipate. She will commit her configuration, share what generalizes, and refine what fits only her. She will, over the months ahead, end up with an environment that no off-the-shelf product could produce, because it has been shaped by her actual work and her actual preferences, and the shaping has been deliberate.

走完本书的读者,到这里手里已经握有:框架、范式、可立即上手的实践库——可以为自己的工作搭出一份有分量的 Claude Code 环境。接下来的每一步,都是她自己的:她会那些"最贴合自己工作"的章节里的组件装上;把另一些按自己偏好改一改;为那些"本书没有料到"的摩擦,从零搭自己的组件;把配置 commit 进去,把"普适的"那部分分享出去,把"只合自己"的那部分精修;在接下来几个月里,一步步长出一份"任何现成产品都给不出来"的环境——因为它是按她实际的工作、实际的偏好被有意识地塑形出来的。

A short word about pacing the build. New readers sometimes feel an urgency to install everything in the book during the first weekend. The urgency is understandable but probably misplaced. The components that have been demonstrated across the chapters were chosen because they cover common needs, but no individual reader needs all of them on day one. The pacing that has worked best in practice is to install a small starter set in the first week, perhaps three skills, two subagents, and two hooks, and then let real use surface which additional components are worth building. A component installed in response to a felt friction is almost always more useful than a component installed because the book described it. The book is a catalog. The reader's real work is the curriculum. Trusting the curriculum, and letting the catalog inform it rather than dictate it, produces an environment that fits the reader's actual work tightly. Trusting the catalog, and forcing the curriculum to match it, produces an environment that fits no actual work in particular.

关于"搭的节奏",说几句。新读者有时会有一股冲动,想在一个周末把书里所有东西都装上。这股冲动可以理解,但多半放错了位置。各章里演示的组件,是按"覆盖常见需求"来挑的——但任何具体读者,都不需要在第一天就全装上。实践中真正管用的节奏是:第一周先装一份"小型启动包"——比如 3 个 skills、2 个 subagents、2 个 hooks——然后让真实使用告诉你"哪些额外组件值得去搭"。

一个"因为感受到了某个摩擦才装"的组件,几乎总比"因为书里描述了它就装"的组件更有用。本书是 catalog;读者真实的工作才是 curriculum。信 curriculum、让 catalog 来充实它而不是支配它,会造出一份紧贴读者实际工作的环境;信 catalog、硬把 curriculum 拗成和它一致,会造出一份"贴合任何具体工作都不太对"的环境。

← 前一页

后一页(收尾)→

The Rhythm of Building · 搭建的节奏

A related observation about the rhythm of building is worth recording. The first month of building is the period in which the most components get added, because the developer is encountering frictions she has noticed for years and finally has the tools to address. The second and third months see fewer additions but more refinement, as the components installed in the first month settle into actual use and reveal their rough edges. By the sixth month, the developer is mostly maintaining and occasionally extending, and the environment has reached a kind of steady state in which it fits her work tightly. This rhythm is normal and healthy. A developer who is still adding components at the same rate after six months has either an unusually broad and complex working life or, more often, has fallen into the over-engineering trap that this book has warned about repeatedly. The right signal that the build is reaching maturity is that new components become rare, refinements become small, and the environment recedes into the background of the developer's daily work, doing its job without drawing attention.

关于"搭建的节奏",还有一条相关的观察值得记下。搭建的第一个月,是"加组件"加得最猛的那段时间——因为开发者正在遇上她多年来注意到、但直到现在才终于有工具去解决的摩擦。第二、第三个月,加的就少了,改的多了——第一个月装上的那些组件,在真实使用中沉淀下来,露出它们粗糙的边。第六个月时,开发者基本进入"维护 + 偶有扩展"的状态;环境已经达到一种稳态,紧密贴合她的工作。这种节奏是正常且健康的。一个开发者,六个月之后还在以同样的速度加新组件——要么她有着异常宽广和复杂的工作生活,要么更常见的,是已经掉进了"过度工程化"那个本书反复警告的坑。"搭建正在走向成熟"那条正确的信号是:新组件变少、精修变小、环境慢慢退到开发者日常工作的背景里去——做它该做的事,不再被注意。

A final word, before this book ends. The developers who invest in their tooling have always had an advantage over the developers who do not. The advantage was real with text editors. It was real with shells. It was real with build systems. It is real now with AI-assisted development environments, and the advantage is, if anything, larger, because the leverage of well-engineered configuration has grown as the tools themselves have grown more capable. The reader who finishes this book and begins her own building has, in the language of investments, made an asset allocation. The asset is her tooling. The expected return is compound, and it begins paying out from the first session in which the configuration affects her work. There is no urgency to build everything at once. There is only the discipline to build deliberately, to use what is built, and to let the environment grow alongside the work it is meant to serve. The reader who follows that discipline will find, in a year or two, that her environment has become something she would not trade for any other configuration in the world, because it has become hers.

本书收尾前的最后一句。在工具上投入的开发者,一直就比不投入的有优势。在文本编辑器上是;在 shell 上是;在 build 系统上是。在 AI 辅助开发环境上仍然如此——而且优势,如果有什么变化的话,是更大了:因为工具本身越来越能打,精心工程化的配置所产生的杠杆也越来越大。读到这里、合上本书、开始自己搭的读者,用投资的话说,已经做完了一笔资产配置。资产就是她的工具;预期收益是复利的;从“配置第一次影响她的工作”那一 session 开始,这份收益就开始兑付了。不必急着一次性搭完;真正要守的纪律只有一条:有意识地搭,把搭好的用起来,让环境跟着它要服务的工作一起长大。守住这条纪律的读者,会在一两年后发现自己手里那份环境,已经是她不愿和世界上任何其他配置交换的东西——因为它已经长成了她。

The book ends here. The work begins.

书到这里收束;工作,才刚要开始。

Appendices · 附录

Reference, Troubleshooting, and Beyond the Three Pillars

参考、排查,以及“三柱之外”

Reference, Troubleshooting, and Beyond the Three Pillars

参考、排查,以及"三柱之外"

The book ended one chapter ago. What follows is what you reach for.

本书正文,一章之前已经收束;接下来的部分,是"你想翻点什么的时候翻"的那部分。

原书第 227 页 · APPENDIX A

227

APPENDIX A · 附录 A

Troubleshooting and Debugging

排错与调试

"The first half of solving a problem is being able to look at it."

— a debugger of long standing, anonymous

"解决问题这件事,前半段在于你能正面看着它。"

—— 一位资深 debugger(匿名)

Real Claude Code environments break in predictable ways. A skill does not fire when it should. A subagent produces generic output that ignores the project's conventions. A hook silently fails to run. A pipeline behaves correctly in one developer's environment and inexplicably differently in another. None of these failures is mysterious once the developer has a debugging procedure in working memory; each of them traces back to one of a small number of root causes, and each of those causes has a clean diagnostic path. This appendix gives the reader the diagnostic paths.

真实的 Claude Code 环境,会以"几种可以预判"的方式坏掉:一个 skill 在该触发时没触发;一个 subagent 产出了忽略项目规约的通用输出;一个 hook 静默地没跑;一条流水线在一个开发者的环

境里跑得对、在另一个开发者的环境里却表现得出奇地不一致。这些失败,只要开发者手头有"一套排查流程"挂在工作记忆里,就没有一个是神秘的——每一种都能回溯到数量不多的几种根因之一,而每一种根因都有一条干净的诊断路径。这份附录,就把这些诊断路径交到读者手里。

The procedures below assume the reader has worked through Parts Two, Three, and Four of the book and is familiar with the underlying mechanics of skills, subagents, and hooks. None of the procedures requires advanced shell knowledge, but each of them does require the reader to be willing to spend a few minutes verifying assumptions one at a time rather than trying to fix everything at once.

下面这套流程,假设读者已经走完本书的第二、三、四部分,熟悉 skill、subagent、hook 的底层机制。每一条流程都不需要高级的 shell 知识;但每一条都需要读者愿意"一次一个、逐项验证假设"——而不是一次性把什么都试着修一遍。

Skill Problems · Skill 类问题

The most common skill failure mode is that the skill does not fire when the developer expects it to. The diagnostic procedure has four checkpoints, and the developer should walk them in order.

Skill 最常见的失效模式是"该触发时没触发"。诊断流程有 4 个检查点;读者应当按顺序走过。

First, verify the skill is installed in a location Claude Code looks at. The two valid locations are the user-level skills directory (`~/ .claude/skills/`) and the project-level skills directory (`.claude/skills/`) at the root of the current project. A skill installed anywhere else will not be discovered.

第一,确认 skill 装在 Claude Code 会去找的位置。两个有效位置:user 级别的 skills 目录 (`~/ .claude/skills/`),以及当前项目根目录下的 project 级别 skills 目录 (`.claude/skills/`)。装在其他位置的 skill,不会被发现。

Second, verify the skill folder contains a file literally named `SKILL.md` , with the exact capitalization. A file named `skill.md` or `Skill.md` will not be discovered on case-sensitive filesystems.

第二,确认 skill 文件夹里恰好存在一个叫 `SKILL.md` 的文件——大小写必须严格一致。在大小写敏感的文件系统上,叫 `skill.md` 或 `Skill.md` 的文件,不会被发现。

Third, verify the YAML frontmatter is well-formed. The frontmatter is the block at the very top of the file fenced by three hyphens on a line of

第三,确认 YAML frontmatter 格式完好。Frontmatter 是文件最顶上、由一行"三个连字符"围起来的那段——

← 前一页

后一页 →

their own. If the fence is missing, indented, or includes characters that the YAML parser rejects, the skill will be silently skipped at session start.

如果围栏丢了、缩进了,或者其中包含了 YAML 解析器会拒绝的字符,那么这个 skill 在 session 启动时会被静默跳过。

Fourth, verify the description field contains language that matches the kind of request the developer is making. This is the failure mode in roughly nine cases out of ten. The fix is to expand the description with the phrasings the developer actually uses, following the four-part framework from Chapter Six (canonical statement, trigger language, boundary, input expectation).

第四,确认 description 字段里装着"和你提出的请求"匹配的语言。十次里有大致九次,问题就出在这里。修法是:把开发者实际会用的那些措辞,按第六章的四段式框架(canonical statement、trigger language、boundary、input expectation)扩进 description。

A useful diagnostic technique when the skill is installed and the description appears reasonable but the skill still does not fire is to ask Claude directly. Type into the session: *why did the [skill-name] skill not fire on my last request*. Claude will often produce a useful answer, naming the gap between the request and the description. That answer is then a precise instruction for what to add to the description. After making the change, restart the session for the change to take effect, and try again. Within two or three iterations, almost every triggering problem can be solved by description refinement.

当 skill 装上了、description 看起来也合理、但就是不会触发——一条很管用的诊断技巧是:直接问 Claude。在 session 里敲:为什么 *[skill-name]* 这个 skill 在我上一条请求上没触发。Claude 通常会给出一条有用的回答,把"请求和 description 之间的差距"点出来。这条回答本身,就是一段精确的"该往 description 里加什么"的指示。改完,重启 session 让改动生效,再试。两三轮之内,几乎所有的触发问题都能被"精修 description"这一招搞定。

A second skill failure mode is over-firing. The skill fires on requests that should not have triggered it. The cause is almost always a description that is too broad, lacking boundary language. The fix is to add explicit exclusions to the description, naming the kinds of requests that look superficially similar to the skill's territory but should not activate it.

Skill 的第二种失效模式是"过触发"——在不该触发的请求上也触发了。根因几乎总是 description 写得太宽,缺了"boundary"那一段。修法是在 description 里加显式的"排除项",把"看起来和这个 skill 的领域表面相似、但不该触发它"的那几类请求点名写出来。

A third failure mode is correct firing but incorrect output. The skill activates as expected, but what it produces does not match what the body specifies. This is almost always a sign that the body is ambiguous in a way the developer did not notice when writing it. The fix is to read the body as if reading it for the first time, find the place where instruction and output diverge, and tighten the language. Side-by-side good and bad examples are extraordinarily effective for fixing this.

第三种失效模式是"触发对了,但输出不对"。Skill 确实按预期激活了,但产出的内容跟 body 的描述对不上。这几乎总是"body 里有某处歧义是写它的人当时没注意到的"那条信号。修法是:像第一次读到这份 body 一样去读它,找到"指令和输出开始分叉"的那一处,把语言收紧。在修这种问题上,"好例子 + 坏例子"并列摆出来,效果异常好。

Subagent Problems · Subagent 类问题

Subagent problems share a structure with skill problems, with one important addition: subagents have a system prompt that does most of the shaping work, and most subagent failures trace back to that system prompt being too vague.

Subagent 的问题,结构上和 skill 的问题是同一族——多一项:subagent 有一个 system prompt 承担了大部分"塑形"工作,大多数 subagent 失败,根因都是"system prompt 太含糊"。

← 前一页

后一页 →

If the subagent does not get dispatched at all, the cause is almost always a description that does not match the request. The diagnostic is identical to the skill triggering procedure. Type the failed request again, then ask Claude why the subagent was not dispatched, and refine the description based on the answer.

如果 subagent 压根就没被派出去,根因几乎总是 description 和请求不匹配。诊断流程和 skill 触发那条一模一样——把失败的请求再敲一次,然后问 Claude 这次为什么没派;再根据它的回答去精修 description。

If the subagent gets dispatched but produces generic output, the cause is almost always a system prompt that gives the subagent too much latitude. The fix is to add specificity: name the method as a numbered procedure, name the output format down to the section headings, and add a *what-you-do-not-do* section that explicitly forbids the failure modes the developer has already encountered.

如果 subagent 被派出去了,但产出的是通用化输出,根因几乎总是 system prompt 给了 subagent 太大的自由空间。修法是:补具体性——把方法用编号化的步骤写出来,把输出格式连"小节标题"都点名定下来,再加一段 *what-you-do-not-do*(你不做什么),把开发者已经踩到过的失效模式显式禁掉。

If the subagent gets dispatched and produces output in the right shape but with the wrong substance, the cause is often a missing skill. The subagent has the role definition but not the project-specific knowledge. The fix is either to add a skill that encodes the project knowledge (which the subagent will then load when relevant) or, less ideally, to put the project knowledge directly into the subagent's system prompt.

如果 subagent 被派出去了,产出"形状对、但内容不对"——根因常常是"缺一条 skill"。Subagent 拿到了"角色定义",但没拿到"项目特有的知识"。修法是:写一条新 skill 把这些项目知识编码进去(子代理会在相关时去加载它);或者退而求其次,把这些项目知识直接塞进 subagent 的 system prompt。

If the subagent does not respect its tool restrictions, the cause is usually a typo in the `tools` field of the frontmatter. The values are case-sensitive and must match the tool names exactly: `Read` , `Write` , `Edit` , `MultiEdit` , `Bash` , `Grep` , `Glob` . A typo silently grants the full tool set.

如果 subagent 不遵守它的工具限制,根因通常是 frontmatter 的 `tools` 字段里某个值打错了。这些值是大小写敏感的,必须和工具名精确匹配: `Read` 、 `Write` 、 `Edit` 、 `MultiEdit` 、 `Bash` 、 `Grep` 、 `Glob` 。一个拼错,会在静默中"开闸"到"全部工具"。

A useful introspection technique is the `/agents` slash command. Run it in a session; it lists every subagent available, shows the description and tool restrictions, and confirms the file is being read correctly. If the developer expects a subagent to exist and `/agents` does not show it, the problem is at the file or YAML level rather than at the description level.

一条很有用的"内省"技巧是 `/agents` 这个 slash command。在 session 里跑一下,它会列出所有可用的 subagent,显示 description 和工具限制,并确认该文件正在被正确读取。如果开发者预期某个 subagent 存在,但 `/agents` 没列出来——那么问题出在文件层或 YAML 层,而不是 description 层。

Hook Problems · Hook 类问题

Hook problems are the most subtle of the three categories, because hooks fail silently more often than they fail loudly. A misconfigured hook does not produce a visible error in most cases; it simply does not run. The four-step diagnostic procedure for hooks is worth committing to memory.

Hook 类问题,是三类问题里最微妙的——因为 hook 经常"静默"地失败,而不是"响亮"地失败。配错的 hook 多数情况下不会产生可见错误,只是不跑而已。Hook 的 4 步诊断流程,值得背下来。

First, verify the hook is loaded. Run the `/hooks` command inside a Claude Code session. The interactive editor lists every event and the hooks bound to each one. If the new hook does not appear, the configuration file

第一,确认 hook 被加载了。在 Claude Code session 里跑 `/hooks` 这个命令。交互式编辑器会列出每一个事件、以及绑定到该事件上的所有 hook。如果新加的 hook 没出现在列表里,那么配置文件——

← 前一页

后一页 →

has a problem, usually a JSON syntax error or a misnamed event. Many hook problems disappear at this step.

本身有问题——通常是个 JSON 语法错误,或事件名写错。很多 hook 问题,走到这一步就消失了。

Second, verify the hook is firing. Add a small log statement to the hook itself, writing a timestamp to a file every time the hook fires. Make an action that should trigger the hook. Check the log file. If the log file has no new entry, the matcher is wrong; the hook is bound to an event or tool name that does not match what is happening. Adjust the matcher and try again.

第二,确认 hook 真的"触发"了。给 hook 自己加一行小日志——每次 hook 触发,就往一个文件里写一个时间戳。然后做一次"本应触发这个 hook"的动作,再去看那个日志文件。如果日志里没新条目,说明 matcher 配错了——hook 被绑到了一个和实际发生动作对不上的事件名或工具名。改 matcher,再试。

Third, verify the hook is succeeding. Read the hook's standard output and standard error in the log. If the hook is exiting with a non-zero code due to a missing tool or an unexpected JSON shape, fix the

cause. A common case is that a tool the hook depends on (like `jq` or `npx`) is not installed on the developer's system, or has been moved out of the path.

第三,确认 hook 是"成功"完成的。读 hook 的标准输出和标准错误日志。如果 hook 因为"依赖工具缺失"或"JSON 形状对不上"而退出码非 0,就去修根因。常见情况是:hook 依赖的某个工具(`jq` 或 `npx` 之类)没装,或者已经不在 PATH 上了。

Fourth, verify the effect of the hook. The hook may be firing, succeeding, and still not producing the intended outcome because of a logic error in the script or because the hook's output is not being interpreted as expected. For hooks that emit JSON to control the lifecycle, validate the JSON shape against the documentation before assuming the hook is broken.

第四,确认 hook 的"效果"。Hook 可能"触发了""成功了",却仍没产出预期结果——根因可能是脚本里的逻辑错,或 hook 的输出没有被按预期解读。对于那些"靠吐 JSON 来控生命周期"的 hook,在怀疑它坏了之前,先按文档核一下 JSON 的形状。

The `/doctor` slash command is worth knowing about as a complementary diagnostic. It reports on the current installation and surfaces common configuration problems including malformed settings files, missing dependencies that hooks rely on, and version mismatches. Running `/doctor` at the start of any debugging session gives the developer a quick read on whether the problem is local to her configuration or environmental.

`/doctor` 这个 slash command 也值得记一下,作为互补的诊断手段。它会报告当前安装状态,主动把"常见的配置问题"摆到台面上:格式有误的 settings 文件、hook 依赖的缺失组件、版本不匹配等。在任何一次排错的开头先跑一遍 `/doctor`,能让开发者快速判断"问题在她自己的配置里"还是"在环境那一层"。

A specific subtlety worth naming: hook configuration changes do not affect a running session. After editing the hooks file or any `settings.json`, the developer must restart Claude Code or open a new session for the change to take effect. Many otherwise-correct hook configurations appear broken because the developer is testing them in the same session in which she made the change.

还有一条具体的"暗坑"值得明说:hook 的配置变更不会影响"已经在跑的 session"。改完 hook 文件或 `settings.json` 之后,必须重启 Claude Code 或开一个新 session,改动才会生效。很

多"看着对、但似乎坏了"的 hook 配置,实际是开发者在"刚改完它的那个 session 里"测的——所以新配置还没生效。

Pipeline Problems · 流水线类问题

When several hooks compose into a pipeline, the diagnostic complexity rises but the procedure does not change. Walk each hook in the pipeline through the four-step procedure individually. The hook that fails one of the four steps is the failing component. Pipelines almost never fail because of

当几个 hook 组合成一条流水线,诊断的复杂度上升了,但流程本身没变。逐个把流水线里的每个 hook 走一遍那 4 步流程——哪一步在哪一个 hook 上挂掉,挂掉那个就是有问题的组件。流水线几乎从来不会"因为"——

← 前一页

后一页 →

pipeline-level issues; they fail because one hook in the pipeline fails, and the failure cascades.

流水线级别的问题而失败;它们失败,是因为流水线里某一个 hook 失败,而失败又沿着链路向下传播。

A second pipeline-specific concern is order. Hooks bound to the same event run in the order they appear in the configuration. If the second hook in a pair depends on the first hook having modified the file system, and the first hook is misordered or misconfigured, the second hook will run on stale state. The fix is to verify the order matches the dependency, and to add a small assertion at the start of any hook that depends on another hook's effect.

流水线专属的第二条关切是顺序。绑在同一事件上的 hook,按它们在配置里出现的顺序执行。如果"成对 hook 里的第二个"依赖"第一个"已经动过文件系统,而第一个的顺序被排错或被配错,第二

个就会跑在陈旧的状态上。修法是:确认"声明顺序"和"实际依赖顺序"一致,并在每个"依赖另一个 hook 副作用"的 hook 开头加一个小断言。

APPENDIX B · 附录 B

Hook Events Quick Reference

Hook 事件速查

"The reference is the part you reread. The argument is the part you read once."
— a textbook author, anonymous

"参考是你回头再读的那部分;正文是你读一遍就够的那部分。"
—— 一位教科书作者(匿名)

This appendix is a compact reference to every hook event Claude Code currently supports, what each event fires for, what input it receives, and the most common pattern of use. The reference is intended to be looked up rather than read straight through. Skim it once to learn what is here, and then return to it when designing a new hook to verify which event fits.

这份附录,是 Claude Code 当前支持的所有 hook 事件的一份紧凑参考:每个事件在什么时点触发、收到什么输入、最常见的使用形态。参考是用来"查"的,不是用来"从头读到尾"的;先通览一遍知道里头有什么,设计新 hook 时再回来核对"哪个事件最贴"。

Event · 事件	Behavior, input, and use · 行为、输入与典型用法
PreToolUse	Fires: 紧接 Claude 调用一个工具之前触发,此时工具名和输入已可用。 Input: JSON 对象,包含工具名、工具输入以及 session 元数据。 Use for: 用于"门"——阻断、修改、或延后工具调用。可用于安全策略强制、危险命令拦截、文件路径保护。

Event · 事件	Behavior, input, and use · 行为、输入与典型用法
	Exit 2: 阻断这次工具调用。标准错误输出会被作为反馈回给 Claude。另外,通过 JSON 输出还能拿到 4 种更丰富的结局:allow / deny / ask / defer。
PostToolUse	Fires: 一个工具调用成功完成后立即触发。 Input: JSON 对象,包含工具名、输入、以及调用结果。 Use for: 用于"守卫"——强制一致性:formatter、linter、edit 后校验、对 Claude 每一个动作的审计日志。 Exit 2: 此刻无法阻断(动作已经发生);标准错误输出会作为反馈回给 Claude,可在下一轮触发修正工作。
UserPromptSubmit	Fires: 用户提交一条 prompt 时,Claude 处理它之前触发。 Input: JSON 对象,包含 prompt 文本和 session 上下文。

← 前一页

后一页 →

UserPromptSubmit (续)	Use for: prompt 级别的过滤、可选地往里注入额外上下文,以及对"哪些请求被允许"做策略强制。 Exit 2: 整条 prompt 被拒。用户会看到拒绝消息。
Stop	Fires: Claude 一次响应生成完毕时触发。 Input: JSON 对象,包含 Claude 准备停下的那一刻的 session 状态。 Use for: 回合末端的校验器、session 摘要写入器、回合末端的审计日志。 Exit 2: 拒绝停。Claude 被强制继续工作,直到一次 stop 真正成功。
SubagentStop	Fires: 一个 subagent 完成任务时触发,刚好在它的结果回到主 session 之前。 Input: JSON 对象,包含 subagent 名字和它的最终输出。 Use for: 把 subagent 的输出按"预期形状"做一遍校验、记录 subagent 的活动。 Exit 2: 拒绝该 subagent 的 stop。请谨慎使用。
SessionStart	Fires: 新 session 启动,或一份被保存的 session 被恢复时触发。 Input: JSON 对象,包含 session 元数据。 Use for: 把"环境级"的项目状态加载进 session 的起始上下文里。 Exit 2: 写到 stdout 的输出会被注入到 Claude 的起始上下文里。

SessionEnd	Fires: 一个 session 终止时触发。 Input: JSON 对象,包含 session 元数据和最终状态。 Use for: session 级别的审计日志、日报生成、清理任务。 Exit 2: 此刻没有"阻断"效果——session 本来就要结束,无论结果如何。
Notification	Fires: Claude 需要提醒用户某件事时触发。 Input: JSON 对象,包含 notification 的 payload。 Use for: 弹桌面通知、把提醒发到聊天平台。 Exit 2: 不会影响生命周期。Notification hooks 是"通知器"型 hook 的规范示例。
PreCompact	Fires: Claude 即将压缩上下文以腾出空间之前触发。

← 前一页

后一页 →

A practical guideline for choosing among these events: the event whose name most directly matches the moment when the work needs to happen is almost always the right one. `PreToolUse` for prevention. `PostToolUse` for correction. `Stop` for end-of-turn validation. `SessionStart` for ambient state injection. With practice, the right event becomes obvious at the moment the developer recognizes which kind of work she is trying to enforce.

在这些事件之间做选择,有一条实用的经验法则:事件名和你想干活的时点最直接对应的那一个,几乎永远是对的。要"事前拦"→ `PreToolUse`。要"事后改"→ `PostToolUse`。要"回合末端校验"→ `Stop`。要"环境级状态注入"→ `SessionStart`。练到一定熟练度,只要意识到"自己想强制的是哪一类工作",对的事件会一眼就跳出来。

PreCompact (续)	Input: JSON 对象,包含当前上下文状态。 Use for: 在重要上下文即将被压掉之前,把它"捞"出来,写一份检查点摘要。 Exit 2: 写到 stdout 的输出会被保留进压缩后的上下文。
----------------	--

注:以上是 Hook 事件速查表的最后一项(`PreCompact` 的 `Input` / `Use for` / `Exit 2` 三个字段)。整张速查表覆盖了 9 个事件: `PreToolUse`、`PostToolUse`、`UserPromptSubmit`、`Stop`、`SubagentStop`、

SessionStart、SessionEnd、Notification、PreCompact。

注:以上是 Hook 事件速查表的最后一项(PreCompact 的 Input / Use for / Exit 2 三个字段)。
整张速查表覆盖了 9 个事件: PreToolUse、PostToolUse、UserPromptSubmit、Stop、
SubagentStop、SessionStart、SessionEnd、Notification、PreCompact。

APPENDIX C · 附录 C

Beyond the Three Pillars: Plugins and Agent Teams

三柱之外:Plugins 与 Agent Teams

"The framework is the foundation. What sits on top of the foundation is the work of the next decade."

— a software architect, anonymous

"框架是地基;建在地基之上的部分,是未来十年要做的工作。"

—— 一位软件架构师(匿名)

The three pillars covered in this book, skills, subagents, and hooks, are the durable core of Claude Code customization, and they are sufficient on their own to build the engineered environments that the case studies in Part Five demonstrated. They are not, however, the entirety of what Claude Code now supports. Two additional capabilities have emerged in the more recent releases that are worth knowing about, both because they extend the architecture in useful directions and because the developer who is fluent in the three pillars will find the additions easy to absorb. This appendix introduces them at a level sufficient to recognize when they apply and to know where to look for deeper coverage.

本书讲过的三柱——skills、subagents、hooks——是 Claude Code 定制化的"耐久核心";单凭这三柱,已经足以搭出第五部分案例里展示的那些工程化环境。但它们并不是 Claude Code 现在支

持的全部。在较新的几次发布里,有两项额外的能力冒出来了,值得知道——一是因为它们在有用的方向上扩展了这套架构,二是因为一位"对三柱已经很熟"的开发者去吸收它们,会发现上手很轻。这份附录把它们介绍到一个"能认出来什么场景该用、并知道去哪里查更深入资料"的水平。

The first additional capability is the plugin system. The second is agent teams.

第一项额外能力,是 plugin 系统;第二项,是 agent teams。

Plugins · Plugins

A plugin, in the Claude Code sense, is a packaged collection of skills, subagents, hooks, and supporting assets that can be installed as a unit. The plugin system formalizes a pattern that this book has already taught informally: the developer who builds a coherent set of components for a specific kind of work is, in effect, building a plugin, even if she has not packaged the components for distribution. The plugin system gives that pattern a formal home.

Plugin,放在 Claude Code 的语境里,就是一份"打包好的集合"——里面装着 skills、subagents、hooks 以及配套的资源——可以作为一个整体来安装。Plugin 系统把本书里已经以一种非正式方式教过的一种范式,正式化了:开发者如果为某类特定工作搭出一组"内部自治"的组件,事实上就等于在搭一份 plugin——即使她还没把这些组件打包成可分发的形式。Plugin 系统,正是给这种范式一个正式的家。

A plugin is a folder that contains a manifest file describing what the plugin provides, the components themselves arranged in their conventional subfolders, and any documentation the author wants to ship with it. The plugin can be installed from a local path, from a Git repository, or from a registry, and once installed it appears alongside the developer's hand-built

Plugin 是一个文件夹:里面有一份 manifest 文件,描述这份 plugin 提供了什么;有按规约子目录摆放的组件本身;再有作者愿意随它一起发出去的文档。Plugin 可以从本地路径装,可以从 Git 仓库装,也可以从一个 registry 装;装上之后,它就和开发者手搭的配置并列出现——

configuration. The plugin's components are activated through the same triggering and dispatch mechanisms covered in Parts Two through Four: skills fire when their descriptions match, subagents are dispatched when their roles fit, hooks bind to lifecycle events. From the model's perspective, a plugin is invisible; the components inside the plugin behave exactly as components installed directly would behave.

和开发者手搭的配置并列出现。Plugin 里的组件,激活方式和第二至第四部分讲过的触发与派发机制一致:description 匹配时 skill 触发,角色对得上时 subagent 被派,生命周期事件和 hook 绑定。从模型的角度看,plugin 是"看不见"的——plugin 里的组件,行为和直接装进环境的组件完全一致。

The strategic value of plugins is twofold. The first value is distribution. A developer who has built a thoughtful environment for a specific niche, such as technical writing, financial analysis, or React development, can package it as a plugin and share it with peers or sell it. The second value is consumption. A developer who recognizes that her work overlaps significantly with an existing plugin can install the plugin and inherit its components, saving herself the time of building the same components from scratch. Both ends of the exchange are useful, and the plugin ecosystem is in the early phase of populating with real, useful packages.

Plugin 的战略价值是双重的。第一重价值是"分发":一位开发者,只要为某个具体 niche(比如技术写作、财务分析、React 开发)搭好了一份深思熟虑的环境,就可以把它打包成 plugin,拿去和同行分享,或者出售。第二重价值是"消费":一位开发者,只要意识到"自己的工作"和某份现成 plugin 高度重叠,就可以把那份 plugin 装下来,继承它的组件,省下自己从零搭同样组件的时间。交换的两端都有用;而 plugin 生态正处在"开始装进真实、有用包"的早期阶段。

The reader who finishes this book and wants to participate in the plugin ecosystem has two natural entry points. The first is to package one of her own most successful sets of components as a plugin and publish it. The second is to browse what is already available, install one or two plugins that fit her work, and observe how the plugin author shaped the components to fit a niche. Either approach develops the

fluency that makes the plugin system a real extension of the developer's environment rather than a separate concern.

走完本书、想要参与 plugin 生态的读者,有两条自然的入口。第一条:把"自己手上最成功的那组组件"打包成一份 plugin,发出去。第二条:去浏览已经上架的那些,挑一两个贴合自己工作的装上,再观察 plugin 作者是怎么把组件塑形去贴合某个 niche 的。两条路,任一条走下去,都会让 plugin 系统从"另一件和你的环境无关的事"变成"你环境的真正延伸"——这种熟稔是练出来的。

Agent Teams · Agent Teams

The second extension is agent teams. The orchestrator pattern from Chapter Twelve introduced the idea of dispatching several subagents in parallel and synthesizing their outputs. Agent teams formalize that pattern. An agent team is a named, configured group of subagents that work together on a single class of task, with explicit coordination logic that specifies how the team communicates and how the final output is assembled.

第二项扩展,是 agent teams。第十二章的 orchestrator 范式讲过一种"并行派出多个 subagent、再综合它们的产出"的做法;agent teams 把这种范式正式化了。一个 agent team 是一组"有名字、配过"的 subagent,它们合作处理某类单一任务;团队里有显式的协调逻辑,规定团队内部怎么通信、最终输出怎么组装。

A team differs from an ad-hoc parallel dispatch in two respects. First, the team is a named, persistent configuration; the developer defines it once and then dispatches the entire team with a single invocation, rather than orchestrating individual subagents on every request. Second, the team can

一支 team 和"临时的并行派发"相比,有两点不同。第一,team 是一个"有名字、可持久化"的配置;开发者只需定义一次,之后用一次调用就能派出整支 team,而不是每次请求都要手工编排那批 subagent。第二,team 可以——

include explicit communication patterns between its members. A simple team is the parallel-fan-out pattern from Chapter Twelve, where each subagent contributes an independent perspective and a synthesizer combines them. A more elaborate team can have sequential dependencies, with one subagent's output becoming the input to the next, or even feedback loops where a reviewer subagent flags issues and a writer subagent revises in response.

包含"成员之间的显式通信模式"。一支简单的 team,就是第十二章的"并行扇出"范式——每个 subagent 提供一份独立的视角,由一个 synthesizer 合成。更复杂一点的 team,可以有"顺序依赖":一个 subagent 的输出成为下一个的输入;甚至可以有"反馈环":reviewer subagent 抛出问题,writer subagent 据此修订。

The team's capability is most valuable for workflows that the developer runs often enough to deserve their own configuration. A weekly review like the one in the solo-founder case study from Chapter Twenty-One is a natural candidate; rather than dispatching the financial-analyst, the marketing-writer, and the product-strategist subagents individually each Friday, the developer can define a weekly-review team that runs all three with a single command and produces the unified review automatically. The development cost of the team configuration is small, and the savings on every future invocation accumulate.

Team 这个能力,对那些"开发者跑得足够频繁、值得为它单写一份配置"的工作流,价值最大。第二十一章独立创业者案例里那份"周五周会回顾",就是天然的候选——与其每周五都单独派 financial-analyst、marketing-writer、product-strategist 这三个 subagent,不如直接定义一支"周会回顾" team:一条命令就把这三个跑完,自动产出一份统一的 review。搭一份 team 配置的开发成本很小;而此后每一次调用省下来的时间,会不断累积。

A reasonable rule of thumb for when to reach for an agent team rather than ad-hoc orchestration: if the same combination of subagents has been dispatched together more than three times in a month, the combination has earned its place as a named team. The team configuration captures the pattern explicitly and removes the small overhead of remembering which subagents to dispatch on every future invocation.

什么时候该伸手去抓"agent team"而不是"临时编排"?有一条合理的经验法则:同一组 subagent,如果一个月内被一起派出去超过 3 次,那这组组合就赢得了"被命名为一支 team"的资格。Team 配

置把这种范式显式记下来,免去了"每次还要想一遍这次该派哪几个 subagent"那一小笔开销。

A final observation is worth making about plugins and agent teams in relation to the rest of the book. Both capabilities sit on top of the three-pillar architecture rather than replacing it. A plugin is a packaged set of skills, subagents, and hooks. An agent team is a coordinated invocation of subagents. The underlying components are the same components covered in Parts Two through Four, and the design disciplines for those components are unchanged. The reader who has internalized the three pillars will find both extensions natural to learn. The reader who has not will find both extensions confusing, because the extensions assume the underlying pillars are already understood. This is the right ordering: pillars first, extensions second, and the extensions become useful precisely because the pillars are solid.

关于 plugins 和 agent teams,值得再点一条观察:它们俩,都是"坐在三柱架构之上",而不是"取代"它。Plugin 是一组"打包好的"skills、subagents、hooks;agent team 是一次"协调过的"subagent 调用。底下的组件,和第二至第四部分讲过的是同一组;对这些组件的设计纪律也没变。已经把三柱内化于心的读者,会发觉这两项扩展都自然到几乎不用学;没把三柱内化于心的读者,则会感到这两项都很让人困惑——因为它们预设了"三柱已经懂了"这个前提。这正是正确的学习顺序:先三柱,后扩展;而扩展之所以"好用",正是三柱"够扎实"这件事带来的回报。

← 前一页

后一页(附录末页)→

Both capabilities are evolving, and the official documentation is the right primary source for current details. The treatment in this appendix is meant to put the capabilities on the reader's mental map and to indicate when reaching for them makes sense. The deeper coverage will mature as the ecosystem matures, and the reader who has finished this book has the architectural foundation to absorb that coverage as it appears.

这两项能力都在持续演化;官方文档,是获取当前细节的"主信源"。本附录对它们的处理,是把它们摆到读者"心智地图"上的相应位置,并指出什么时候伸手去抓它们是合适的。更深入的覆盖,会随生态一起成熟;而读完了本书的读者,手里已经有吸收那些新覆盖所需的架构基础。

APPENDIX D · 附录 D

Key Terms Index

关键术语索引

"A book without an index is a city without street signs."

— a librarian, anonymous

"没有索引的书,是一座没有路牌的城市。"

—— 一位图书管理员(匿名)

The index below is a curated list of the technical terms and architectural concepts the book has used most often, with a brief gloss for each one. The index is intended to serve as a quick lookup when the reader encounters a term in a later chapter and wants to recall its meaning, or when she returns to the book months later and needs to refresh her vocabulary. The list is alphabetical and the glosses are short on purpose; for the full treatment of any term, the reader should consult the chapter where the term was introduced.

下面的索引,是本书出现频率最高的技术术语和架构概念的一份精挑过的清单,每条都附有简短的释义。这份索引的用途,是供读者速查——当她在后面某一章碰到一个术语、想唤起它的含义,或者几个月后再回来、需要"刷一遍词汇表"的时候。本表按字母排序,释义刻意写得短;任何术语的完整论述,请回查"该术语最初出现的那一章"。

Agent loop. The think-act-observe cycle that the language model executes on every turn. Skills and subagents shape what happens inside the loop; hooks shape what happens at its boundaries. See Chapter Two and Chapter Fifteen.

Agent loop(代理循环)。 语言模型每一轮都执行的"思考—行动—观察"循环。Skills 和 subagents 塑形的是循环内部发生的事;hooks 塑形的是循环边界上发生的事。参见第二章和第十五章。

Agent team. A named, persistent configuration of subagents that work together on a single class of task, with coordination logic specifying how the team communicates. See Appendix C.

Agent team(代理小队)。 一组"有名字、可持久化"的 subagent 配置,合作处理某类单一任务,内部有协调逻辑规定通信方式。参见附录 C。

allowed-tools . The optional frontmatter field in a `SKILL.md` file that restricts the set of tools Claude can use while the skill is active. Reaches its purpose by narrowing the model's capability surface to exactly what the skill needs. See Chapter Four.

allowed-tools (允许工具列表)。 `SKILL.md` 文件里一个可选的 frontmatter 字段,用来限制该 skill 激活期间 Claude 可用的工具集合。它的作用是"把模型的能力面收窄到正好是这条 skill 需要的那个范围"。参见第四章。

Ambient versus targeted knowledge. The architectural distinction between knowledge that is true at all times in a project (which belongs in `CLAUDE.md`) and knowledge that is true only when a specific kind of work is being done (which belongs in a skill). See Chapter Two and Chapter Eight.

环境级 vs 触发级(Ambient versus targeted)。 一条架构上的区分:"一个项目里任何时候都为真的知识"(归 `CLAUDE.md`)vs "只在某类特定工作进行时才为真的知识"(归 skill)。参见第二章和第八章。

.claude/ . The convention directory at the root of a project, or in the user's home folder, that holds Claude Code configuration. Inside it live the `skills/` and `agents/` subdirectories, the `settings.json` file that configures hooks,

.claude/ 目录。放在项目根目录、或者用户 home 目录的"约定目录",Claude Code 的配置就住在那里。里面住着 `skills/` 和 `agents/` 子目录、用来配 hooks 的 `settings.json` 文件,

← 前一页

后一页 →

and (by convention) a `hooks/` folder for the shell scripts that `settings.json` references. See Chapter Two.

以及(按约定)一个 `hooks/` 目录,装着 `settings.json` 引用的那些 shell 脚本。参见第二章。

C

CLAUDE.md . The special file Claude Code reads at session start whose contents are added to the system context. The right place for ambient project knowledge that should apply to every interaction. See Chapter Two.

CLAUDE.md 。 Claude Code 在 session 启动时读入的一个特殊文件,文件内容会被加进系统上下文。它是"项目级环境知识、应当作用于每一次交互"这件事的归属地。参见第二章。

Composition pattern. The architectural pattern in which skills activate inside subagents, producing capability and specialization simultaneously. The deepest leverage in the three-pillar framework. See Chapter Fourteen.

组合范式(Composition pattern)。一条架构范式:让 skill 在 subagent 内部被激活,从而同时获得"能力"和"专门化"。三柱框架里最深层的杠杆所在。参见第十四章。

D

Description. The frontmatter field in any skill or subagent definition that determines whether the component fires or is dispatched on a given request. The single most consequential design decision in any component. See Chapter Six.

Description(描述字段)。任一 skill 或 subagent 定义里 frontmatter 中的字段,决定该组件在面对某条给定请求时是否触发或被派。任何组件里"后果最严重的一条设计决策"。参见第六章。

Deterministic outer / probabilistic inner. The architectural pattern in which the language model operates probabilistically inside the agent loop, while hooks impose deterministic constraints at the loop's boundaries. The central design principle of the book. See Chapter Three.

外层确定 / 内层概率(Deterministic outer / probabilistic inner)。一条架构范式:让语言模型在 agent loop 内部按概率性运行,而 hooks 在 loop 的边界上施加确定性约束。本书的中心设计原则。参见第三章。

G

Gate. A hook design pattern in which a `PreToolUse` hook evaluates a tool call and either allows or blocks it. The mechanism by which wishes become rules. See Chapter Sixteen.

Gate(门)。一种 hook 设计范式: `PreToolUse` hook 评估一次工具调用,选择放行或阻断。"愿望变成规则"的那条机制。参见第十六章。

Guard. A hook design pattern in which a `PostToolUse` hook runs after a tool call and enforces consistency on its result. The mechanism by which conventions become automatic. See Chapter Sixteen.

Guard(守卫)。一种 hook 设计范式: `PostToolUse` hook 在一次工具调用之后跑,对其结果强制

一致性。"规约变成自动"的那条机制。参见第十六章。

H

Hook. A user-defined shell command that Claude Code executes automatically at a specific lifecycle event. The third pillar of the framework, providing deterministic enforcement. See Part Four.

Hook(钩子)。 Claude Code 在某个特定生命周期事件上自动执行的一条用户自定义 shell 命令。框架里的第三柱,提供确定性强制。参见第四部分。

hookSpecificOutput . The JSON output field in a hook that carries richer control instructions to Claude Code, including the four `PreToolUse` outcomes (allow, deny, ask, defer) and the modified tool input. See Chapter Fifteen.

hookSpecificOutput (hook 专属输出)。 一条 hook 里负责把"更丰富的控制指令"送回 Claude Code 的 JSON 输出字段,包括四种 `PreToolUse` 的结局(allow、deny、ask、defer)以及被修改后的工具输入。参见第十五章。

N

Notifier. A hook design pattern in which a hook produces output to a system other than Claude Code itself, such as a desktop notification or an audit log entry. See Chapter Sixteen.

Notifier(通知器)。 一种 hook 设计范式:让 hook 把输出送到"Claude Code 自己之外"的某个系统,比如桌面通知、审计日志条目。参见第十六章。

← 前一页

后一页 →

O

Orchestrator pattern. The multi-subagent pattern in which the main session decomposes a task, dispatches several subagents in parallel, and synthesizes their outputs into a unified result. The most common pattern of subagent composition. See Chapter Twelve.

Orchestrator 范式。 一条多 subagent 范式:主会话拆解一个任务,并行派多个 subagent,再把它们的输出合成为一份统一结果。subagent 组合里最常用的一种形态。参见第十二章。

P

Plugin. A packaged collection of skills, subagents, hooks, and supporting assets that can be installed as a unit. The formal mechanism for distributing a coherent set of components. See Appendix C.

Plugin(插件)。 一份"打包好的"skills、subagents、hooks 以及配套资源的集合,可作为一个整体来安装。分发"一套内部自治组件"的正式机制。参见附录 C。

Progressive disclosure. The loading strategy in which Claude Code reads only the descriptions of available skills at session start, then loads a skill's body only when the skill activates, and only reads bundled files when the body refers to them. The mechanism that lets large skill libraries scale without paying a context tax. See Chapter Four.

渐进式披露(Progressive disclosure)。 一种加载策略:Claude Code 在 session 启动时只读所有可用 skill 的 description;只有在某条 skill 真正激活时,才去读它的 body;而它打包进来的文件,只在 body 引用到时才读。这条机制让大型 skill 库能扩展规模、而不用付"上下文税"。参见第四章。

S

Skill. A reusable capability defined by a folder containing a `SKILL.md` file. The first pillar of the framework. See Part Two.

Skill(技能)。一种"可复用的能力",由一个装着 `SKILL.md` 文件的目录来定义。框架里的第一柱。参见第二部分。

SKILL.md . The Markdown file inside a skill folder that contains YAML frontmatter declaring the skill's metadata and a body containing the instructions Claude follows when the skill is active. See Chapter Four.

SKILL.md 。 skill 文件夹里的那份 Markdown 文件,顶上是一段声明该 skill 元数据的 YAML frontmatter,正文是"该 skill 激活时 Claude 要遵循的那些指令"。参见第四章。

Subagent. A separate Claude process with its own system prompt, restricted tool set, and isolated context window, dispatched by the main session for specific kinds of work. The second pillar of the framework. See Part Three.

Subagent(子代理)。一个独立的 Claude 进程:自己的 system prompt、受限的工具集、被隔离的上下文窗口,由主会话派去处理特定种类的任务。框架里的第二柱。参见第三部分。

T

Three-pillar framework. The book's organizing architecture: skills for capability, subagents for specialization, hooks for enforcement. The framework that any sophisticated Claude Code environment uses in some combination. See Chapter Three.

三柱框架(Three-pillar framework)。本书整套论述的组织性架构:skills 负责"能力",subagents 负责"专门化",hooks 负责"强制"。任何"严肃的 Claude Code 环境"都会以某种组合形式用到它。参见第三章。

@

@agent-name . The explicit invocation syntax for a subagent. Typing `@agent-name` in a prompt dispatches the named subagent immediately, bypassing description-based delegation. See Chapter Ten.

@agent-name 语法。 显式调用 subagent 的语法:在 prompt 里敲 @agent-name ,被点名的那个 subagent 会被立刻派出去——绕过"按 description 派发"那条路径。参见第十章。

The terms above cover the vocabulary the book has been building since Chapter One. Reading the index in one pass is a quick way to confirm that the central concepts have settled into working memory. Returning to it later, when one of the terms surfaces in a new context, is how the index earns its place in a reference book. The reader who has finished the book has, by this

上面这些术语,覆盖了本书从第一章开始一直在构建的那套词汇表。一遍通读,就能快速确认那些核心概念是否都已落入工作记忆。以后某个术语在新的语境下冒出来时,再回来翻——这就是一份"参考书里的索引"该有的存在方式。读完本书的读者,到这里——

← 前一页

全书末页 →

point, a working command of every term above; the index is a backstop for the moments when one of them needs to be looked up rather than recalled.

已经对上面每一条术语都"可工作地掌握"了;这份索引,只是在"某条术语需要被查、而不是被想出来"那些时刻的"兜底"。

— 全书完 —